Reviewer Pack — Coxeter Group Enumeration of Boolean Function Entropy via In…

Entry 027781f6d598 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 01:21:02 UTC

Coxeter Group Enumeration of Boolean Function Entropy via Invariant Generators

Entry ID: `027781f6d598` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 01:21:02 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Group Theory (specifically, Coxeter groups) **Field B** (complexity object): Boolean function entropy

Statement:

> For every boolean function f: {0,...,n-1} -> {-1,1}, let χ(f) be the number of distinct irreducible representations of the Coxeter group Γ_f associated with f. Then, for all instances with n ≤ 40, χ(f) = Θ(2^n).

Rationale (proposer's reasoning):

> Coxeter groups provide a rich algebraic structure that can categorize the symmetries in boolean functions. The invariant generators of these groups may capture the complexity of the function's entropy, providing a new perspective on computational hardness.

Taxonomy category: `COXETER_GROUP_ENTROPY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8283b34f2a7062f6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For boolean functions f with n ≤ 40, χ(f) will be considered supported if $|\chi(f) - 2^{n/2}n \leq 0.05$ for all n and at least 80% of seeds, and falsified if any seed has $|\chi(f) - 2^{n/2}n > 0.1$ or χ(f) ≠ Θ(2^n).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group" AND "Boolean function entropy" - "irreducible representations" AND "Coxeter groups" AND "boolean functions" - "entropy of boolean functions" AND "Coxeter group enumeration"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_boolean_function(n):
    return [random.choice([-1, 1]) for _ in range(2**n)]

def conjugacy_class_enumeration(f):
    n = int(math.log2(len(f)))
    classes = set()
    for i in range(n):
        class_set = {j for j in range(2**n) if f[j] == f[j ^ (1 << i)]}
        classes.update(class_set)
    return classes

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 20, 40]:
        instances_tested = 0
        n_max = 0
        total_metric_value = 0
        conjecture_holds = True
        counterexample = ""

        for _ in range(30):
            f = generate_boolean_function(n)
            classes = conjugacy_class_enumeration(f)
            chi_f = len(classes)
            instances_tested += 1
            n_max = max(n_max, n)

            if instances_tested == 1:
                expected_value = 2**n
```

```
                tolerance = 0.05 * expected_value
                if abs(chi_f - expected_value) / expected_value > tolerance:
                    conjecture_holds = False
                    counterexample = f"chi(f)={chi_f}, expected={expected_value}"

            total_metric_value += chi_f

        mean_value = Fraction(total_metric_value, instances_tested)

        results.append({
            "metric_name": "chi(f)",
            "metric_value": float(mean_value),
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        })

    return {
        "seed": seed,
        **results[0]
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] * r["instances_tested"] for r in results)
    mean_value = Fraction(total_metric_value, sum(r["instances_tested"] for r in results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(abs(r["metric_value"] - 2**r["n_max"]) / 2**r["n_max"] > 0.1 or not r["conjecture_hol
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if abs(result["metr
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it did not meet the pre-registered support condition for χ(f) to be considered supported. | next: Run the test again with increased time limits or optimize the code to ensure it completes within the allowed time frame.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14467
2	propose	ollama_remote	glm4:latest	0	0	27407
3	preregistration	ollama_remote	glm4:latest	0	0	9882
4	novelty	ollama_remote	glm4:latest	0	0	19297
5	novelty	ollama_remote	glm4:latest	0	0	11950
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13261
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15306
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	121051
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14218
10	judge	ollama_remote	glm4:latest	0	0	32383

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 279222 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/027781f6d598.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/027781f6d598.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/027781f6d598.tar.gz` (if generated)